

Software Engineering
Tutorial 3 - The design patterns

PhD. Ha Manh Hung

Exercice 1

The code of the classes `CompteBancaire` and `Journalisation` and of the `main` function is given below :

```
#pragma once
#include <memory>

#include "Journalisation.hpp"

class CompteBancaire {
private:
    int numero;                         // Account number
    double solde;                       // Money available in the account
    std::shared_ptr<Journalisation> journalisation;

public:
    // Constructor
    CompteBancaire(int numero);

    // Deposit money
    void deposerArgent(double depot);

    // Withdraw money
    void retirerArgent(double retrait);
};
```

```
#include <iostream>

#include "CompteBancaire.hpp"

#include <string>

CompteBancaire::CompteBancaire(int numero)
    : numero(numero),
      solde(0.0), journalisation(std::make_shared<Journalisation>()) {
        std::cout << journalisation.get() << std::endl;
}

void CompteBancaire::deposerArgent(double depot) {
    if (depot > 0.0) {
        solde += depot;

        journalisation->afficherLog(
```

```
            "Dépôt de " + std::to_string(depot) +
            "€ sur le compte " + std::to_string(numero) + "."
        );
    }
}

void CompteBancaire::retirerArgent(double retrait) {
    if (retrait > 0.0) {
        if (solde >= retrait) {
            solde -= retrait;

            journalisation->afficherLog(
                "Retrait de " + std::to_string(retrait) +
                "€ sur le compte " + std::to_string(numero) + "."
            );
        }
        else {
            journalisation->afficherLog(
                "La banque n'autorise pas de découvert sur le compte " +
                std::to_string(numero) + "."
            );
        }
    }
};
```

```
#pragma once

#include <string>

class Journalisation {
public:
   void afficherLog(const std::string& log);
};
```

```
#include "Journalisation.hpp"

#include <iostream>
#include <chrono>
#include <ctime>
#include <iomanip>

void Journalisation::afficherLog(const std::string& log){
   // Retrieving the current date and time
   auto maintenant = std::chrono::system_clock::now();
   std::time_t temps = std::chrono::system_clock::to_time_t(maintenant);

   std::tm* date = std::localtime(&temps);
   std::cout << "["
          << date->tm_mday << "/"
          << date->tm_mon + 1 << "/"
          << date->tm_year + 1900
          << " at "
          << date->tm_hour << "h"
          << date->tm_min
          << "] "
          << log
          << std::endl;
}
```

```cpp
#include "CompteBancaire.hpp"

int main()
{
    // Creation of BankAccounts cb1, cb2, cb3.
    std::shared_ptr<CompteBancaire> cb1 =
                                    std::make_shared<CompteBancaire>(123456789);
    std::shared_ptr<CompteBancaire> cb2 =
                                    std::make_shared<CompteBancaire>(987654321);
    std::shared_ptr<CompteBancaire> cb3 =
                                    std::make_shared<CompteBancaire>(456789012);

    // Using the BankAccount cb1.
    // TO BE COMPLETED

    // Using the BankAccount cb2.
    // TO BE COMPLETED

    // Using the BankAccount cb3.
    // TO BE COMPLETED

    return 0;
}
```

Question 1 : Complete the `main` function so that the following result is displayed in a console :

```
[19/8/2026 at 11h01] Deposit of 100.000000€ into account 123456789.
[19/8/2026 at 11h01] Withdrawal of 80.000000€ from account 123456789.
[19/8/2026 at 11h01] The bank does not allow overdrafts on account 987654321.
[19/8/2026 at 11h01] Deposit of 100.000000€ into account 456789012.
```

Question 2 : How many objects of type Journalisation are created during the execution of the `main` function of the Main class ?

We now want to implement the `Journalisation` class based on the `Singleton` pattern, whose general class diagram is shown in Figure 1.

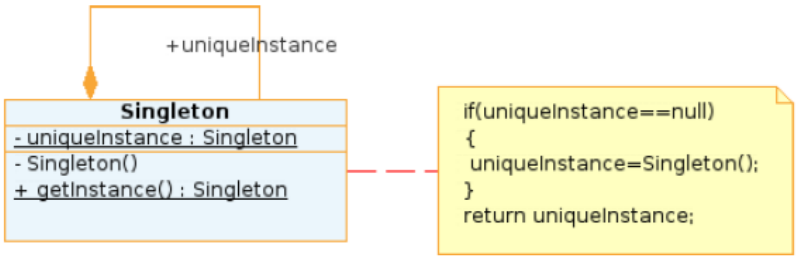


FIGURE 1 – Class diagram of the Singleton pattern.

Question 3 : Provide the class diagram of `Journalisation`. Explain how it works.

Question 4 : Provide the code for the `Journalisation` class. Modify the code of the `CompteBancaire`

class so that it works with the `Journalisation` class. Test the new solution.

Question 5 : How many objects of type `Journalisation` are created during the execution of the `main` function.

Exercice 2

The concept of event logging (or logging in English) consists in adding processing to an application that allows messages to be generated and stored following particular events. For example, a trace of the exceptions that occurred during the execution of the application can be kept in a file. The messages recorded in the file can be used during the debugging phase of the application or when it is deployed in production.

We want to integrate into an existing application a feature that keeps track of abnormal events that occurred during its execution. The messages generated by the application will be displayed in a console or stored in a file as simple text : `Message d'erreur n° 1`. The messages can also be formatted or encrypted before being displayed in a console or stored in a file.

Formatting a message simply consists in enclosing it in HTML tags :

```
<HTML><BODY> <b> Message d'erreur n° 1 </b> </BODY> </HTML>
```

Encrypting a message consists in shifting each character of the message n positions to the left. The message `Message d'erreur n° 1` shifted 4 positions to the left gives the following message : `age d'erreur n° 1Mess`.

The class diagram in Figure 2 can be used to implement this functionality.

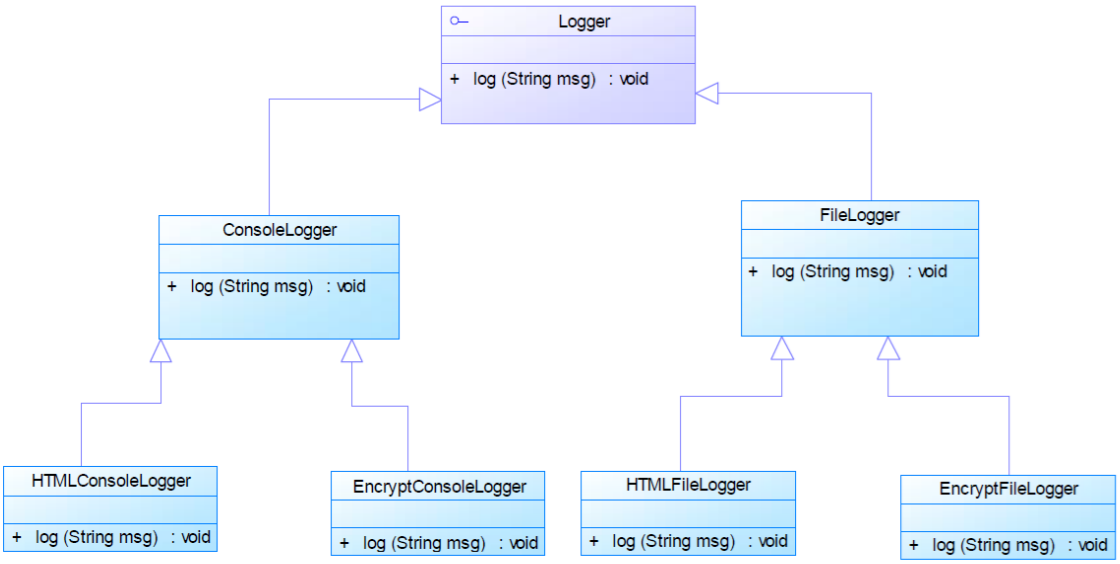


FIGURE 2 – Class diagram for managing the event history in an application.

Question 1 : Comment on the proposed solution.

The class diagram in Figure 3 describes the structure of the Decorator design pattern. This design

pattern makes it possible to dynamically add additional properties to an object while limiting the need to define new classes. It provides an interesting alternative to multiple inheritance.

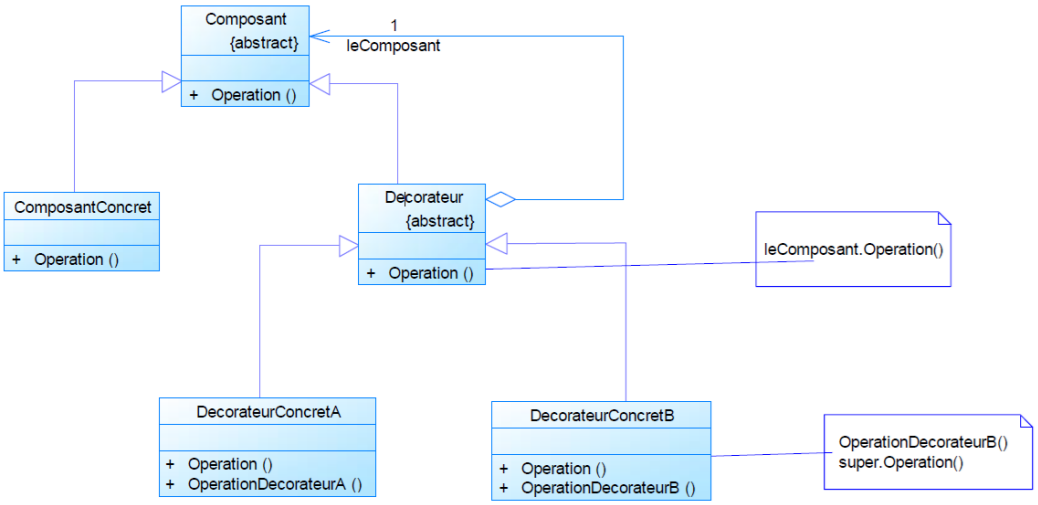


FIGURE 3 – Class diagram of the Decorator pattern.

The following classes can be identified :

- the Composant class, which defines a common interface for all classes that inherit from it,
- the ComposantConcret class, which makes it possible to instantiate an object to which additional responsibilities can be attached,
- the Decorateur class, which maintains a reference to the Composant class and defines an interface that conforms to that of the component,
- the DecorateurConcretA and DecorateurConcretB classes, which make it possible to add responsibilities to a component and define their own operations, OperationDecorateurA and OperationDecorateurB.

Question 2 : Propose a class diagram using the Decorator design pattern and allowing messages to be logged.

Question 3 : Provide the object diagram showing the objects that are created when the following instructions are executed :

```
FileLogger fLogger = new FileLogger();
HTMLLogger hLogger = new HTMLLogger(fLogger);
```

Question 4 : Provide the sequence diagram showing the messages exchanged between the objects when messages are stored in a text file in HTML format.

Question 5 : Provide the C++ code corresponding to the class diagram given in Question 2. You may use the code of the `Logger`, `FileLogger` and `ConsoleLogger` classes given below :

```
#pragma once
```

```cpp
#include <string>

class Logger {
public:
   virtual ~Logger() = default;
   virtual void log(const std::string& msg) = 0;
};
```

```cpp
#pragma once

#include <iostream>
#include <fstream>

#include "Logger.hpp"

class FileLogger : public Logger {
private:
    std::string nomFichier_;

public:
    FileLogger(const std::string nomFichier) : nomFichier_(nomFichier) {
    }

    void log(const std::string& msg) override {
        std::ofstream fichier(nomFichier_, std::ios::app);

        if (fichier.is_open()) {
            fichier << msg << std::endl;
            fichier.close();
        }
        else {
            std::cerr << "Erreur : impossible d'ouvrir le fichier "
                    << "FichierLog.txt" << std::endl;
        }
    }
};
```

```cpp
#pragma once

#include "Logger.hpp"

class ConsoleLogger : public Logger {
public:
   void log(const std::string& msg) override;
};
```

```cpp
#pragma once

#include "Logger.hpp"

#include <iostream>

class ConsoleLogger : public Logger {
public:
    void log(const std::string& msg) override {
        std::cout << msg << std::endl;
    }
};
```

Question 6 : Test your solution with the `main` function given below :

```
#include "ConsoleLogger.hpp"
#include "EncryptLogger.hpp"
#include "FileLogger.hpp"
#include "HTMLLogger.hpp"

#include <string>
#include <memory>

int main() {
    // Case 1: linking an object of type EncryptLogger to an object of type
    // ConsoleLogger
    // Creating an object of type ConsoleLogger
    std::shared_ptr<ConsoleLogger> cLogger1 = std::make_shared<ConsoleLogger>();

    // Creating an object EncryptLogger
    std::shared_ptr<EncryptLogger> eLogger1 =
                                    std::make_shared<EncryptLogger>(cLogger1);

    // We generate 10 log messages that are encrypted
    // and then displayed in a console.
    for (int i = 0; i < 10; i++) {
        eLogger1->log(std::to_string(i) + " : A message to log");
    }


    // Case 2 : linking an object of type HTMLLogger to an object of type
    // FileLogger
    // Creating an object of type FileLogger
    std::shared_ptr<FileLogger> fLogger1 =
                                std::make_shared<FileLogger>("FichierLog1.html");

    // Creating an object of type HTMLLogger
    std::shared_ptr<HTMLLogger> hLogger1 = std::make_shared<HTMLLogger>(fLogger1);

    // We generate 10 log messages that are formatted
    // in HTML and then saved to a file.
    for (int i = 0; i < 10; i++) {
        hLogger1->log(std::to_string(i) + " : A message to log");
    }
    return 0;
}
```